# LOF CRM – Test, Security & API Handover

Oct 12, 2026 · @Mushtaq

Version 2 (counsellor targets, payment terms, refund rules, LMS integration) passed all 143 automated tests and 12 browser tests after four review rounds; it is ready for Cloudflare staging, then production once credentials and the business decisions below are confirmed.

## Status at a glance

| Area | Result |
|---|---|
| Automated tests (API, sync, integrations, security, units) | 143 / 143 passed (version 2) |
| Browser tests (real screens against the Worker) | 12 / 12 passed (version 2) |
| TypeScript type check | Clean |
| Production build (wrangler deploy --dry-run) | OK – 290 KiB, 75 KiB gzipped |
| Independent security reviews | 3 rounds; every high or critical finding fixed, each with a regression test |
| Integrations | All 10 adapters tested in mock mode and against stubbed provider APIs; live mode turns on per integration once its credentials are set |
| Hosted pilot (Claude artifact) | Paused – it cannot meet this security level (no server-side checks). Use Cloudflare staging instead |

## Test report

Vitest runs the real Worker code (same Hono app, sync rules and adapters) on an in-memory SQLite database built from the four production migrations. Playwright drives the real screens in Chromium against wrangler dev with a freshly migrated local D1.

```
npm test             # 120 tests, ~20 s
npm run test:e2e     # 10 browser tests, ~40 s
npm run typecheck
```

## Results by suite

| Suite | Tests | Covers |
|---|---|---|
| test/unit | 17 | HMAC, JWT, PBKDF2, TOTP, fee maths, templates, Moodle encoding, Exotel/Meta/WhatsApp parsing, log redaction, live/mock switching |
| test/api/smoke | 1 | App boots, health check, migrations apply |
| test/api/auth-and-sync | 23 | Sign-in, lockout, password rules, role preview, branch-scoped bootstrap, every sync guard, approvals, sequences |
| test/api/rest | 6 | Lead API with routing and duplicates, scoping, student 360, receipts, payment links |
| test/api/create-admin | 1 | Admin script stores only a hash and forces a password change |
| test/integrations/webhooks | 12 | Website, Meta, WhatsApp, Exotel, Razorpay, Telr – verification and idempotency |
| test/integrations/live-adapters | 8 | Exact requests to Moodle, WhatsApp, Exotel, ERPNext, Telr against stubbed APIs, including provider errors |
| test/integrations/testing-console | 11 | Readiness, connection tests, 4 end-to-end scenarios, webhook simulations, cleanup, scheduler |
| test/security/security | 29 | Round-1 review findings plus sessions, CSRF, two-step verification, headers |
| test/security/round2 | 12 | Round-2 and round-3 findings: issued IDs, append-only history, refund cap, permissions, sign-in hardening, log redaction |
| Total | 120 | All passed |

## Browser tests

| # | Flow | Result |
|---|---|---|
| 1 | Counsellor creates a lead; saved on the server with routing | Passed |
| 2 | Counsellor cannot open Configuration | Passed |
| 3 | Administrator runs the UAE scenario from the Testing console | Passed |
| 4 | Sign-out clears all data from the page; screen keeps working | Passed |
| 5 | Reception records a counter payment and gets a server receipt number | Passed |
| 6 | Centre Administrator registers a new student; Moodle sync completes | Passed |
| 7 | Management approves a pending exceptional discount | Passed |
| 8 | System Administrator saves SLA configuration | Passed |
| 9 | Counsellor cannot see another branch’s leads | Passed |
| 10 | System Administrator issues a temporary password; user must change it at sign-in | Passed |

## Testing console scenarios (in the app)

Settings → Testing console (System Administrator only); every step passes:

- UAE enquiry → routing → opportunity → enrolment → Telr payment link → paid → receipt → Moodle → ERPNext
- India enquiry → same flow with INR and Razorpay
- Meta Lead Ads import, including retry de-duplication
- Moodle outage → failure recorded → automatic retry succeeds

Test records use TST- numbers, never consume business numbering, are hidden from other users and are removed with Delete test data.

# Issues found and fixed

## Functional issues (testing and QA)

| Issue | Fix |
|---|---|
| Long pop-up forms (e.g. New lead) pushed the Save button off-screen | Pop-ups scroll inside; footer stays visible |
| Centre Administrator's first payment in the enrolment form was rejected | Allowed as part of a new enrolment |
| Routing found no counsellor when a branch was full; test leads counted toward workload | Capacity check in scenarios; test leads excluded from workload |
| Two people editing the same lead could overwrite each other | Field-level merge: each person's changed fields are kept; the user is told when a colleague changed the same field |
| Bulk assign showed the wrong count; Lost leads could not be reopened | Fixed in the screens |
| Leftover "simulated" wording on screens and in sample data | Replaced with accurate wording; sample data cleaned |
| After sign-out the previous user's data stayed in browser memory; clicks could error | Sign-out clears all data and timers; handlers guard a signed-out state |
| No way for an administrator to issue staff passwords | Staff & user management → Edit: Issue temporary password, + reset two-step, Sign out everywhere |
| Production seed gave every staff login the public demo password | Masters-only seed sets random unknown passwords; the administrator issues temporary ones |
| Staging environment had no database or queue bindings (deploy would fail) | Separate staging D1 and queue in wrangler.toml |

## Security findings

Three independent review rounds tried to break the system from outside (forged requests, other roles, other branches). Every high and critical finding is fixed and covered by a test.

| Severity | Finding | Fix |
|---|---|---|
| Critical | Sync accepted unknown operations, skipping the guards | Only upsert and delete; record IDs validated and must match |
| High | Records could be moved into your branch, or a lead moved without an approved transfer | Branch scope checked on old and new version; transfers must match the approval exactly |
| High | Client could set fees, discounts and totals | Fee recomputed from the fee master; discounts must be predefined or an approved exceptional discount for the same opportunity, programme and fee, usable once |
| High | Payments: overpayment, backdating, client receipt numbers, editing | Amount ⩽ balance; date clamped; server receipt numbers; DB triggers block overpayment and any change to payments or audit |
| High | Client-built approvals, messages, Moodle records, notifications | Rebuilt on the server: approver from configuration, validated payload and recipients; client text never sent as-is |
| High | Session token readable by page scripts (localStorage) | HttpOnly __Host- cookie, SameSite=Strict, CSRF header on writes, server-side sessions, 60-min idle timeout |
| High | Forged record numbers (IDs never issued by the server) | New records must use numbers issued by the server sequence |
| High | History (events, assignments, approval history) could be rewritten | Append-only; the server stamps who and when |
| High | Refund total could exceed what was paid | Cumulative refund cap |
| High | An unchecked edit in the same save could unlock another rule (e.g. raise paid to lift the refund cap) | Rules read related records as stored; records checked in dependency order |
| High | Testing console open to all configuration roles | System Administrator only |
| High | Payment links had guessable IDs; mock links could be marked paid in production | Random 10-character link IDs; mock paid only on test links or outside production; atomic settlement |

| Severity | Finding | Fix |
|---|---|---|
| Medium | No two-step verification | TOTP authenticator apps, required for System Administrator and Management |
| Medium | Sign-in enumeration and brute force | One generic error; per-IP and per-e-mail limits; lockout; dummy hash for unknown e-mails |
| Medium | Sign-in could be posted from another site | JSON only; a browser Origin must be this app |
| Medium | Student 360 showed payments to roles without Payments access | Payments, enrolments, Moodle data returned only with the matching permission and branch |
| Medium | REST lead creation accepted any field | Field allow-list; counsellor must be active in that branch |
| Medium | Secrets could appear in integration logs | Every configured secret value redacted |
| Medium | Test records could reach live providers in production | Test records always mock in production |
| Low | No rate limit on two-step disable and sequence blocks | Added |
| Low | Unescaped values in receipts; weak headers | HTML escaping; CSP with script hashes, HSTS, frame denial |

# Security controls

## Identity and sessions

- Passwords: PBKDF2-SHA256 (100,000 iterations, random salt); at least 10 characters with 3 of 4 character classes; no e-mail name or common words; forced change at first sign-in and after an administrator reset.
- Two-step verification (TOTP): any authenticator app; secrets AES-GCM encrypted; codes cannot be reused; required for roles in MFA_REQUIRED_ROLES (default System Administrator, Management). Until a required user enrols they reach only the set-up screens.

- Sessions: random token in an HttpOnly, Secure, SameSite=Strict __Host- cookie; the server stores only its SHA-256 hash; ends after 60 minutes idle or 12 hours; all of a user's sessions end on password change, two-step change or administrator reset; an administrator can sign anyone out everywhere.
- CSRF: every cookie-authenticated write must send x-lof-csrf: 1; CORS limited to the app's own origin.
- Brute force: sign-in 10 per e-mail and 30 per IP per 15 min; 5 failures lock the account 15 min; two-step and password-change attempts limited; 600 API requests per minute per user.
- Role preview (impersonation) is impossible in production.

## Authorization: every write re-checked on the server

Each save (PUT /sync) is all-or-nothing and checks:

| Check | Example |
|---|---|
| Role permission (Roles & Permissions matrix) | Reception cannot edit leads; only configuration roles change settings |
| Branch scope, old and new version | A Karama counsellor cannot read, move or edit Sports City records |
| Approvals | Branch change only through a matching approved transfer; approver must be the configured role and not the requester; a decided request cannot change |
| Server-issued numbers | New LD/OP/EN/PY/LOF/AP/RF numbers must have been issued by the server |
| Append-only history | Timeline, assignment and approval history only grow; server stamps who and when |
| Fee integrity | Fee recomputed from the fee master; discounts validated; exceptional approval usable once |
| Money | Payment ⩽ balance, configured methods, reference for non-cash, server receipt numbers; refunds capped at the amount paid |
| Consent | WhatsApp or e-mail without consent stored as Blocked and never sent |
| Size limits | 64 KB per record; 500 changes and 2 MB per save |

## Data protection

- Database triggers as the last line of defence: no overpayment; payment amount, enrolment, currency and receipt number cannot change; payments and audit entries cannot be deleted; audit entries cannot be edited.
- Least data: staff e-mail and mobile hidden from roles without configuration access; payments hidden without Payments → View; integration credentials never sent to the browser.
- Logs: every provider request and response logged with all secret values, tokens and authorisation headers redacted.
- Retention (automatic): integration logs 30 days; webhook IDs 90 days; ended sessions, rate counters and call screen-pop events shortly after they expire.

### Web and transport

- Content-Security-Policy: inline scripts allowed only by exact SHA-256 hash; connect-src 'self'; frame-ancestors 'none'; the API answers with default-src 'none'.
- HSTS (1 year), X-Frame-Options DENY, nosniff, strict Referrer-Policy, Permissions-Policy, COOP; Cache-Control no-store on API responses.
- The Worker refuses to run if JWT_SECRET is shorter than 32 characters.

### Webhooks

| Provider | Verification |
|---|---|
| Website form | HMAC-SHA256 signature or allowed Origin; rate limit, honeypot, field caps, optional Cloudflare Turnstile |
| Meta, WhatsApp | X-Hub-Signature-256 (constant-time compare); verify token |
| Exotel | Secret token; optional IP allow-list (EXOTEL_ALLOWED_IPS) |
| Razorpay | X-Razorpay-Signature; event ID de-duplication |
| Telr | Every payment confirmed server-to-server with Telr's check API; scheduler reconciles open orders |

All webhooks are idempotent.

### Administrator account

mustaq@toprockinteriors.com is created with scripts/create-admin.mjs (runbook step 5). The password is read from an environment variable and stored only as a PBKDF2 hash – not in the code, the seed or the package. Change it at first sign-in: it was shared in chat, so treat it as exposed. The account must also set up two-step verification at first sign-in.

## Residual risks and recommendations

| # | Item | Recommendation |
|---|---|---|
| 1 | JWT_SECRET also encrypts two-step secrets; rotating it disables everyone's two-step | Rotate only with a plan: reset two-step for all users afterwards |

| # | Item | Recommendation |
|---|---|---|
| 2 | Masters-only seed staff use the prototype's example e-mails | Replace with real staff (Configuration → Staff) before go-live, then issue temporary passwords |
| 3 | The screens keep the prototype's workflow code; a rule added only to the screens is not enforced | Add each new business rule to src/sync/sync.ts too, with a test |
| 4 | Rate limits are per database, fixed window | Optionally add Cloudflare WAF rate-limit rules on /api/v1/auth/* and /webhooks/* |
| 5 | Inline style attributes need style-src 'unsafe-inline' | Low risk – scripts are hash-locked |
| 6 | No backup policy configured yet | Keep D1 Time Travel (30 days) and schedule a weekly wrangler d1 export |
| 7 | Refund rules (eligibility, pro-rata) not automated | Confirm the policy (Decisions, #2) |
| 8 | No external penetration test yet | Run a short external test against staging before go-live |

## Deployment runbook (Cloudflare)

Prerequisites: a Cloudflare account (Workers Paid plan recommended for queue volume and CPU limits), Node 20+, npm install in the project folder, npx wrangler login.

1. Staging first – runs every integration in mock mode with role preview; sign in with the demo users (Welcome@LOF2026, must be changed) and run every Testing console scenario.
2. Production database and queues.
3. Production data: masters only – no demo records, no known passwords.
4. Settings and secrets: set APP_BASE_URL in wrangler.toml to the real URL (webhook and payment return URLs are built from it), set JWT_SECRET, then each integration's secrets as it goes live.
5. Administrator account.
6. Deploy, then add the custom domain under Workers → lof-crm → Settings → Domains & Routes.
7. First sign-in: change the password, scan the two-step QR code, replace the example staff with real people, and for each person use Edit → Issue temporary password (give it in person or by phone).

8. Verify: GET /health returns ok; Testing console shows readiness; run the scenarios, then Delete test data.

```
# 1. Staging
npx wrangler d1 create lof_crm_staging            # id into [env.staging] in
wrangler.toml
npx wrangler queues create lof-crm-jobs-staging
npm run build
npx wrangler d1 migrations apply lof_crm_staging --remote --env staging
openssl rand -base64 48 | npx wrangler secret put JWT_SECRET --env staging
npx wrangler deploy --env staging

# 2. Production database and queues
npx wrangler d1 create lof_crm                    # database_id into
wrangler.toml
npx wrangler queues create lof-crm-jobs
npx wrangler queues create lof-crm-jobs-dlq

# 3. Production data (masters only)
node scripts/patch-frontend.mjs
node scripts/build-seed.mjs --masters-only
npm run db:migrate:remote

# 4. Server secret
openssl rand -base64 48 | npx wrangler secret put JWT_SECRET

# 5. Administrator account
ADMIN_PASSWORD='<your temporary password>' node scripts/create-admin.mjs \
  --email mustaq@toprockinteriors.com --name "Mushtaq" > admin.sql
npx wrangler d1 execute lof_crm --remote --file admin.sql
rm admin.sql

# 6. Deploy
npm run deploy
```

Rollback: npx wrangler rollback returns to the previous Worker version; restore data with D1 Time Travel (npx wrangler d1 time-travel restore lof_crm --timestamp=…).

One-command alternative: ./deploy.sh (macOS/Linux) or .\deploy.ps1 (Windows) prompts for the Cloudflare account ID, API token and administrator details, runs the type check and tests, and then runs scripts/cf-deploy.mjs. Plain-language steps for non-developers are in DEPLOY.md in the code package.

### Checking the data

- Settings → Database (System Administrator): every table with row counts, search, branch and test-record filters, full record view, Export CSV and Full backup (JSON). Read-only; passwords, two-step secrets and session tokens are never shown; exports and backups are rate-limited and written to the audit log.
- SQL: npx wrangler d1 execute lof_crm --remote --command "SELECT * FROM v_instalments WHERE due_date < date('now')" – views v_leads, v_opportunities, v_students, v_enrollments, v_instalments, v_payments, v_refunds, v_lms_calls, v_approvals, v_tasks, v_calls, v_staff, v_targets, v_audit. The Cloudflare dashboard (Storage & Databases → D1 → lof_crm → Console) runs the same queries in the browser.
- Restore a JSON backup: node scripts/restore-backup.mjs backup.json > restore.sql, then npx wrangler d1 execute lof_crm --remote --file restore.sql, then delete restore.sql. D1 Time Travel (30 days) remains the first choice after a mistake.

## Integration go-live checklist

For each integration: set its secrets (npx wrangler secret put NAME), set its variables in wrangler.toml, enable it under Configuration → Integrations, then use Test connection and the scenarios in the Testing console. It stays in mock mode until enabled with all secrets set.

| Integration | Secrets / variables | Register with the provider |
|---|---|---|
| Website form | WEBSITE_FORM_SECRET, optional TURNSTILE_SECRET; allowed origins in Configuration | Embed public/embed/lead-form.js, or POST JSON to /webhooks/website with X-LOF-Signature |
| Meta Lead Ads | META_APP_SECRET, META_VERIFY_TOKEN, META_PAGE_ACCESS_TOKEN | Webhook (Page → leadgen) → /webhooks/meta; map form IDs to branch and programme |
| WhatsApp | WHATSAPP_PHONE_NUMBER_ID, WHATSAPP_ACCESS_TOKEN, WHATSAPP_APP_SECRET, WHATSAPP_VERIFY_TOKEN | Webhook → /webhooks/whatsapp (messages); approved template names on each CRM template |

| Integration | Secrets / variables | Register with the provider |
|---|---|---|
| Exotel | EXOTEL_SID, EXOTEL_API_KEY, EXOTEL_API_TOKEN, EXOTEL_CALLER_ID, EXOTEL_WEBHOOK_TOKEN, EXOTEL_SUBDOMAIN, optional EXOTEL_ALLOWED_IPS | Passthru → /webhooks/exotel/incoming?token=…; status callback → /webhooks/exotel/status?token=…; mobile number on each counsellor |
| LMS (LOF booking platform) | LMS_URL, LMS_TOKEN; Track/Credit codes and branch codes in Settings → LMS Integration | REST web service with the 5 functions listed in the README; manual enrolment on courses; idnumber = LOF Student ID |
| Telr (AED) | TELR_STORE_ID, TELR_AUTH_KEY, TELR_TEST_MODE | Optional transaction advice → /webhooks/telr |
| Razorpay (INR) | RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET | Webhook → /webhooks/razorpay (payment_link.paid, .cancelled, .expired) |
| ERPNext | ERPNEXT_URL, ERPNEXT_API_KEY, ERPNEXT_API_SECRET, ERPNEXT_COMPANY, ERPNEXT_PAID_TO_AED, ERPNEXT_PAID_TO_INR | API user with Accounts User role; entries created as drafts unless submit is on |
| Google Calendar | GCAL_SERVICE_ACCOUNT_JSON, GCAL_CALENDAR_ID | Share the calendar with the service account |
| E-mail (Resend) | RESEND_API_KEY, EMAIL_FROM | Verify the sending domain |

## Decisions to confirm with the business

| # | Decision | Current default |
|---|---|---|
| 1 | First payment inside the enrolment form | Allowed for roles that can enrol, even with view-only Payments |

| # | Decision | Current default |
|---|---|---|
| 2 | Refund policy (eligibility, pro-rata, fees retained) | Refund rules (Settings → Refund Rules) are placeholders: 100% ⩽ 7 days/2 sessions, 75% ⩽ 30/6, 50% ⩽ 60/12, 25% ⩽ 90/18, admin fee AED 100 / INR 1,500, registration retained; exceptions approved by Management |
| 3 | SLA business hours | Not applied; the SLA clock runs continuously |
| 4 | INR → AED rate for reports | Manual setting |
| 5 | Receipts vs tax invoices | CRM issues receipts; tax invoices come from ERPNext |
| 6 | Counsellor-initiated reassignment | Goes to the Admissions Head for approval |
| 7 | Roles that must use two-step verification | System Administrator and Management (MFA_REQUIRED_ROLES) |
| 8 | Session idle timeout | 60 minutes (SESSION_IDLE_MINUTES) |

# API reference

Base URL https://<host>/api/v1; JSON in and out (content-type: application/json). The same reference ships as docs/API.md in the code package.

Authentication, two ways:

- Browser (the CRM screens): POST /auth/login with header x-lof-csrf: 1 sets an HttpOnly, SameSite=Strict, Secure session cookie (__Host-lof_session). Every cookie write (POST/PUT/PATCH/DELETE) must also send x-lof-csrf: 1; page scripts never see the token.
- API clients and integrations: POST /auth/login without that header returns { token }; send Authorization: Bearer <token>.

Sessions are stored server-side (SHA-256 hash only), end after 60 minutes idle (SESSION_IDLE_MINUTES) or 12 hours, and are revoked on logout, password change, two-step change and administrator reset.

Errors return { error, details? }: 400 invalid · 401 not signed in · 403 permission, branch scope or a pending sign-in step (details.code = password_change or mfa_setup) · 404 · 409 conflict, rejected sync, duplicate or unissued record number · 413 too large · 415 not

JSON · 422 business rule · 429 rate limited. Lists take limit (1–200, default 50) and offset and return { total, limit, offset, items }, always limited to the caller's branches and permissions.

## Authentication and account security

| Method | Path | Body | Notes |
|---|---|---|---|
| POST | /auth/login | { email, password, code? } | → { user, stage, mustChangePassword, mfaSetupRequired, token? }. With two-step on and no code → { mfaRequired: true }; resend with the 6-digit code. JSON only (415); foreign Origin refused (403); one generic error; 5 failures lock 15 min; 10 per e-mail and 30 per IP per 15 min (429) |
| POST | /auth/logout | – | Ends this session, clears the cookie |
| GET | /auth/me | – | { user, stage, mfaEnabled, mfaRequired } |
| POST | /auth/change-password | { current, next } | ≥ 10 characters, 3 of 4 classes, no e-mail name or common words; signs out other sessions; 10 attempts / 15 min |
| POST | /auth/mfa/setup | – | → { secret, otpauth } for the authenticator QR; stored AES-GCM encrypted |
| POST | /auth/mfa/enable | { code } | Turns two-step on; signs out other sessions; 10 attempts / 10 min |
| POST | /auth/mfa/disable | { password, code } | Not for roles in MFA_REQUIRED_ROLES; 10 attempts / 10 min |
| POST | /auth/reset-password | { staffId, resetMfa? } | System Administrator → { temporaryPassword }; must be changed at next sign-in; signs the user out everywhere |
| POST | /auth/revoke-sessions | { staffId } | System Administrator: sign a user out of every device |

| Method | Path | Body | Notes |
|---|---|---|---|
| POST | /auth/impersonate | { staffId } | Role preview for System Administrators, only when ALLOW_IMPERSONATION=true and not production |

Sign-in stages: a session that still needs a password change or two-step enrolment can call only /auth/*; everything else answers 403 with details.code. Rate limits per user: all API calls 600/min, /seq/claim 120/min, /seq/block 60/min; website form 10 per 10 min per IP.

## Screens (state and sync)

| Method | Path | Notes |
|---|---|---|
| GET | /bootstrap | Branch-scoped state for the screens: { me, server, data: { leads, opps, students, enr, pay, …, cfg, perms }, seq } |
| PUT | /sync | { changes: [{ coll, id, op: upsert or delete, doc, base? }], settings: [{ key, doc }] } → { ok, docs, notices, settings, jobs } or 409 { error, rejected }. All-or-nothing; ⩽ 500 changes and 2 MB per call, 64 KB per record. base = the version the user started from; only that user's changed fields are applied, with notices when a colleague changed the same field. New records must use numbers from /seq/*. History lists are append-only and stamped by the server |
| POST | /seq/claim | { name, expect } → { ok, value } – gapless numbers (lead, opp, enr, pay, student, ap, rf, pd) |
| POST | /seq/block | { name, size } → { start, size } – internal numbers (task, call, cm, notif, audit, ms) |

## Leads and sales

| Method | Path | Notes |
|---|---|---|
| GET | /leads | Filters: status, branch, counsellor, category, source, from, to, q |
| GET | /leads/:id | { lead, tasks, calls, comms, opportunities } |

| Method | Path | Notes |
|---|---|---|
| POST | /leads | { student, parent, phone, email?, branch, prog, src, camp?, cat?, age?, dob?, school?, grade?, curr?, wa?, em?, notes?, cslr?, allowDuplicate? } – only these fields are used; cslr must be an active counsellor of that branch → 201 { lead, assignedTo, duplicates }. Applies routing, queues the automated WhatsApp/e-mail, creates the first-call task; 409 with duplicates if phone or e-mail exists |
| POST | /leads/duplicates | { phone, email } → { duplicates } (other branches masked) |
| POST | /leads/:id/click-to-call | Exotel: rings the signed-in user's mobile, then the parent → { ok, mode, callSid } |
| GET | /calls/incoming?since=ISO | Screen-pop events for the user (from the Exotel passthru) |
| GET | /tasks | status, overdue=1; counsellors get their own |
| GET | /opportunities | stage |
| GET | /approvals | status, type |

## Students, enrolments, payments

| Method | Path | Notes |
|---|---|---|
| GET | /students | q (ID, name, parent, phone, e-mail) |
| GET | /students/:id | { student, enrollments, payments, moodle } – enrolments and Moodle only with Enrolments → View, payments only with Payments → View, limited to your branches |
| GET | /enrollments | status |
| GET | /payments | from, to |
| GET | /payments/:id/receipt | Printable HTML receipt |
| POST | /payments/links | { enrollmentId, amount? } → { id, provider: telr or razorpay, mode, url, amount, currency }; defaults to the outstanding balance |

| Method | Path | Notes |
|---|---|---|
| GET | /payments/links?enrollmentId= | Links and status (created, paid, expired, cancelled, failed) |
| POST | /payments/links/:id/send | { channel: WhatsApp or Email } – sends the link to the parent (consent applies) |

## LMS calls, instalments, refunds, targets,, notifications, audit

| Method | Path | Notes |
|---|---|---|
| GET | /lms/calls | status |
| GET | /lms/calls/:id | – |
| POST | /lms/calls/:id/retry | Runs the sync now → updated sync record |
| GET | /notifications | Filtered for the role → { items, unread } |
| POST | /notifications/read | { ids } |
| GET | /audit | ref (Audit logs → View) |
| GET | /lms/preview/:enrollmentId | { enrol, cancel, missing, mode } – exactly what the LMS calls would send. /moodle/syncs remains an alias of /lms/calls |
| GET | /enrollments/:id/instalments | Payment term and each instalment with paid, balance and status (Paid, Due, Partly paid, Upcoming, Overdue, Closed) |
| GET | /refunds/suggest?enrollmentId=&sessions= | Suggested refund from the refund rules with the breakdown and the approver |
| GET | /targets?month=YYYY-MM | Counsellor targets and achievement in your branches (a counsellor gets their own row) |

## Testing console (/testing, System Administrator only)

| Method | Path | Notes |
|---|---|---|
| GET | /testing/status | Readiness of each integration |
| POST | /testing/integrations/:key/test | website, meta, whatsapp, exotel, lms, telr, razorpay, erpnext, gcal, email |
| POST | /testing/webhooks/:kind/simulate | website, meta, meta-verify, whatsapp-verify, whatsapp-status, whatsapp-inbound, exotel-incoming, exotel-status, telr-paid, razorpay-paid, bad-signature |
| POST | /testing/scenarios/:name/run | uae-lead-to-enrolment, india-lead-to-enrolment, meta-lead, lms-failure-retry → { ok, steps: [{ step, ok, detail, ms }] } |
| GET | /testing/logs, /testing/logs/:id | integration, ok=0, mode, limit |
| POST | /testing/flags | { moodleFail: true } – mock failure injection |
| POST | /testing/cron | Run the scheduler now |
| POST | /testing/cleanup | Delete all test data |

## Database (/admin/db, System Administrator only, read-only)

| Method | Path | Notes |
|---|---|---|
| GET | /admin/db/tables | Datasets with row counts |
| GET | /admin/db/rows | name, q, branch, test=1, limit ⩽ 500, offset → columns, rows, total |
| GET | /admin/db/rows.csv | Same filters, ⩽ 50,000 rows, formula-safe; 30/hour; audited |
| GET | /admin/db/record | name, id → full record and meta |
| GET | /admin/db/backup | lof-crm-backup/1 JSON; 5/hour; audited |

## Webhooks (public, authenticated by signature or token)

| Path | Provider | Authentication |
|---|---|---|
| POST /webhooks/website | Website form | X-LOF-Signature (HMAC-SHA256 hex) or allowed Origin |
| GET/POST /webhooks/meta | Meta Lead Ads | hub.verify_token; X-Hub-Signature-256 |
| GET/POST /webhooks/whatsapp | WhatsApp Cloud API | hub.verify_token; X-Hub-Signature-256 |
| GET/POST /webhooks/exotel/incoming | Exotel passthru | ?token= (optional IP allow-list) |
| POST /webhooks/exotel/status | Exotel status callback | ?token= |
| POST /webhooks/razorpay | Razorpay | X-Razorpay-Signature |
| POST /webhooks/telr | Telr transaction advice | Confirmed server-to-server with Telr's check API |
| GET /webhooks/pay/return | Customer return page | Status re-checked with the gateway |

All webhooks are idempotent (provider event IDs recorded) and logged with secrets redacted; signatures are compared in constant time. GET /health → { ok, db, env, time } for uptime monitoring.